

Absolutely! Here's a **complete Selenium test suite** using pytest that demonstrates **all major Pytest annotations** on the site: <https://www.saucedemo.com>.

What This Covers

Pytest Feature	Demonstrated
<code>@pytest.fixture</code>	Reusable browser setup
<code>@pytest.mark.parametrize</code>	Login with multiple credentials
<code>@pytest.mark.smoke</code>	Tagging core test
<code>@pytest.mark.sanity</code>	Tagging feature test
<code>@pytest.mark.dependency</code>	Test dependency setup
<code>@pytest.mark.order</code>	Control test execution
<code>@pytest.mark.skip</code>	Skipping a test
<code>@pytest.mark.xfail</code>	Expected failure
<code>pytest.ini</code>	Marker registration

Folder Structure

```
saucedemo_tests/  
├── conftest.py  
├── test_login.py  
├── pytest.ini  
└── requirements.txt
```

conftest.py – Browser Fixture

```
import pytest
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager

@pytest.fixture
def driver():
    service = Service(ChromeDriverManager().install())
    driver = webdriver.Chrome(service=service)
    driver.maximize_window()
    yield driver
    driver.quit()
```

pytest.ini – Marker Registration

```
[pytest]
markers =
    smoke: Core test for system stability
    sanity: Focused feature test
    regression: Full coverage test
    login: Login-related tests
```

test_login.py – All Annotations in Action

```
import pytest
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

URL = "https://www.saucedemo.com/"

def login(driver, username, password):
    driver.get(URL)
    driver.find_element(By.ID, "user-name").clear()
    driver.find_element(By.ID, "password").clear()
    driver.find_element(By.ID, "user-name").send_keys(username)
    driver.find_element(By.ID, "password").send_keys(password)
    driver.find_element(By.ID, "login-button").click()
```

```

# SMOKE test with dependency and order
@pytest.mark.smoke
@pytest.mark.login
@pytest.mark.dependency(name="valid_login")
@pytest.mark.order(1)
def test_valid_login(driver):
    login(driver, "standard_user", "secret_sauce")
    WebDriverWait(driver, 5).until(
        EC.presence_of_element_located((By.CLASS_NAME, "inventory_list"))
    )
    assert "inventory" in driver.current_url

# SANITY test that depends on successful login
@pytest.mark.sanity
@pytest.mark.dependency(depends=["valid_login"])
@pytest.mark.order(2)
def test_inventory_access(driver):
    assert "inventory" in driver.current_url

# PARAMETERIZED sanity test
@pytest.mark.sanity
@pytest.mark.parametrize("username,password", [
    ("invalid_user", "secret_sauce"),
    ("standard_user", "wrong_pass"),
    ("", "")
])
@pytest.mark.order(3)
def test_login_invalid_cases(driver, username, password):
    login(driver, username, password)
    error = driver.find_element(By.XPATH, "//h3[@data-test='error']").text
    assert "Epic sadface" in error

# SKIPPED test
@pytest.mark.skip(reason="Feature under development")
@pytest.mark.order(4)
def test_cart_checkout(driver):
    pass # Simulating a placeholder for future implementation

# XFAIL test (known bug)
@pytest.mark.xfail(reason="Locked user shouldn't access the system")
@pytest.mark.order(5)
def test_locked_out_user(driver):
    login(driver, "locked_out_user", "secret_sauce")
    assert "inventory" in driver.current_url # This should fail

```

Run Commands

Run all tests:

```
pytest -v
```

Run only smoke tests:

```
pytest -v -m smoke
```

Run sanity tests:

```
pytest -v -m sanity
```

Bonus: Requirements File (requirements.txt)

```
selenium  
pytest  
pytest-order  
pytest-dependency  
webdriver-manager
```

Install:

```
pip install -r requirements.txt
```

Summary

Annotation	Example Use
<code>@pytest.fixture</code>	WebDriver setup/teardown
<code>@pytest.mark.smoke</code>	Core login test
<code>@pytest.mark.sanity</code>	Invalid login scenarios
<code>@pytest.mark.parametrize</code>	Data-driven login tests
<code>@pytest.mark.dependency</code>	Ensure test runs only if another passes
<code>@pytest.mark.order(n)</code>	Runs tests in intended sequence
<code>@pytest.mark.skip</code>	Placeholder for future feature
<code>@pytest.mark.xfail</code>	Expected to fail due to known issue (bug/limitation)